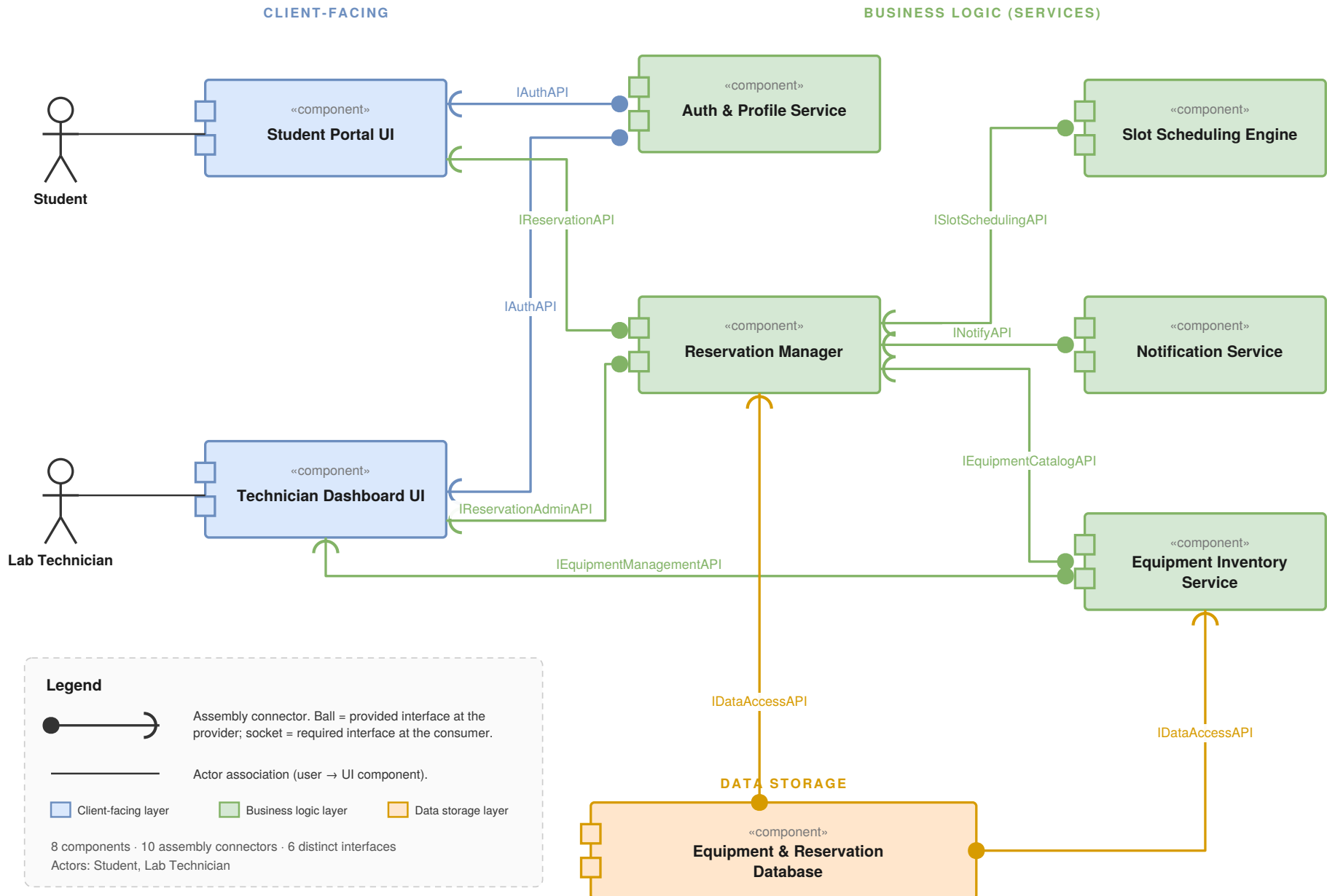


Smart Lab Equipment & Slot Reservation Portal

UML Component Diagram — Microservices Architecture



Lab 3 – Architectural Justification

Smart Lab Equipment & Slot Reservation Portal

Architecture Selection: “We chose Microservices Architecture for the Smart Lab Equipment & Slot Reservation Portal.”

Architectural Choice

The system is decomposed into five independently deployable services — Auth & Profile Service, Reservation Manager, Equipment Inventory Service, Slot Scheduling Engine, and Notification Service — fronted by two client components: the Student Portal UI and the Technician Dashboard UI. Each service exposes a well-defined interface and owns its own logic, consistent with a Microservices architectural style.

Two Reasons for This Choice

1. Independent scaling for bursty, predictable load: Reservation and Notification traffic spikes sharply at the start of every semester and before lab-heavy deadlines, while Equipment Inventory and Auth traffic stays comparatively flat. Microservices let the Reservation Manager and Notification Service scale out independently during these windows without over-provisioning the rest of the system.

2. Fault isolation between core booking and secondary concerns: If the Notification Service (e.g., an email/SMS gateway) becomes slow or unavailable, students should still be able to browse equipment and confirm reservations. Because each service is an independently deployable unit with its own process boundary, a failure in Notification does not cascade into Reservation Manager or Equipment Inventory Service — unlike a monolithic layer where all functionality shares one failure domain.

Security Advantage

Centralizing authentication and authorization in a dedicated Auth & Profile Service means every other service, and both client UIs, enforce access control through one well-tested provided interface (IAuthAPI) instead of re-implementing checks locally. This narrows the attack surface, lets role-based permissions (Student vs. Lab Technician) be audited and updated in a single place, and prevents a compromise in one service from directly exposing credentials or reservation data, since services communicate only through explicit, scoped interfaces rather than a shared session layer.

Performance Benefit

Because Equipment Inventory Service and Reservation Manager are separate services with their own data access paths to the shared database, read-heavy equipment catalog lookups (students browsing availability) do not contend with write-heavy reservation transactions on the same execution path, as they would in a single-layer monolith. The Slot Scheduling Engine can also be scaled and optimized independently for its more CPU-bound conflict-detection workload, keeping browsing latency low even during peak booking periods such as the first week of a semester.